

AlphaTracer – Slimme Aandelenportefeuille & Koersalarmering

App Store-tekst (commercieel)

Volg, analyseer en bescherm je beleggingen – alles in één app.

"AlphaTracer geeft je realtime marktinzichten, een intelligente portefeuillemanager en aanpasbare koersalarmering. Of je nu een beginnende of ervaren belegger bent, je mist geen enkele koersbeweging."

Waarom AlphaTracer installeren?

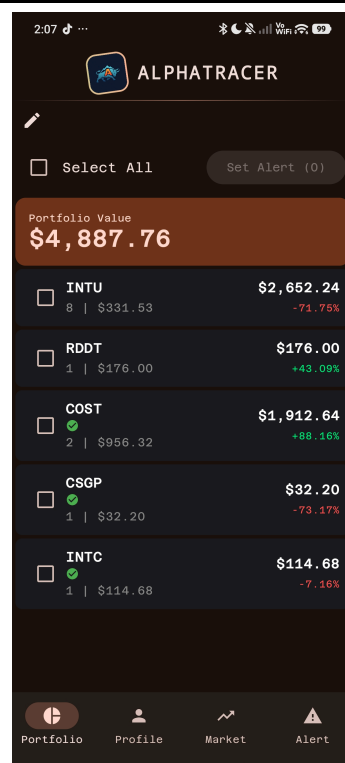
- **Live marktdata** – Zoek elk aandeel en bekijk gedetailleerde financiële cijfers, technische analyse en handelssignalen.
- **Portefeuillebeheer** – Voeg aan- en verkooptransacties toe, zie totale waarde, winst/verliespercentages en volg je prestaties in één oogopslag.
- **Slimme alarmeren** – Ontvang een melding wanneer een aandeel een bepaald percentage daalt over een glijdend venster (bijv. laatste 3 dagen). Stel alarmeren individueel of in bulk in.
- **Veilig en gemakkelijk** – Biometrische login (vingerafdruk/gezicht) houdt je gegevens veilig. Je sessie blijft actief dankzij automatische tokenvernieuwing.
- **Moderne, vloeiende interface** – Gebouwd met Jetpack Compose, donker thema en interactieve grafieken.

Screenshots

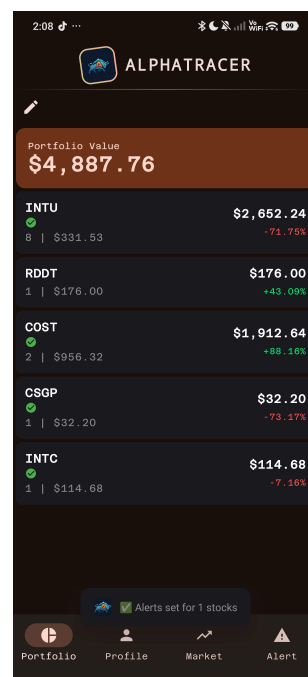
Dashboard



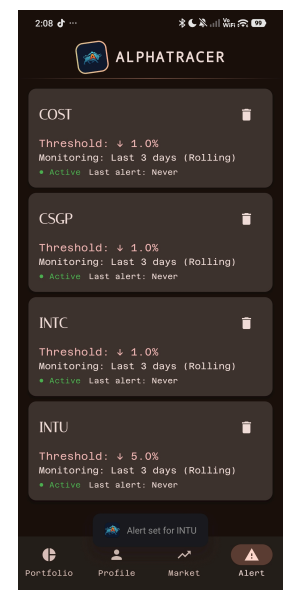
Aandeeldetails



Portefeuille



Alarmeren



 Architectuuroverzicht

AlphaTracer volgt het **MVVM-patroon (Model-View-ViewModel)** met een duidelijke scheiding van verantwoordelijkheden.

Projectstructuur (Android Studio)

```
app/src/main/java/com/main/alphatracer/
├── auth/                                # Authenticatie & tokenbeheer
│   ├── AuthScreen.kt
│   └── Modulair/
│       ├── BiometricAuthenticator.kt
│       └── TokenManager.kt
├── data/                                # Repository-laag
│   ├── PortfolioRepository.kt
│   └── StockRepository.kt
├── model/                               # Dataklassen (AlertRule, Transaction, etc.)
├── network/                             # Retrofit API-configuratie
│   ├── ApiService.kt
│   └── RetrofitClient.kt
├── ui/                                  # Jetpack Compose-schermen + ViewModels
│   ├── Alert/                           # Alarmoverzicht, Worker, DataStore
│   ├── market/                          # Aandelen zoeken
│   ├── Portfolio/                       # Portfoliolijst, bulkalarmen, selectiemodus
│   ├── StockUi/                         # Aandeeldetails, grafieken, koop/verkoopdialoog
│   ├── theme/                           # Material 3 thema & typografie
│   ├── user/                            # Profielscherm
│   └── ViewModel/                       # MainViewModel (navigatie, gedeelde state)
├── MainActivity.kt                      # Startpunt, onderste navigatie, scaffold
└── (AndroidManifest.xml, resources)
```

Belangrijkste componenten

Component	Verantwoordelijkheid
Composables	AuthScreen, PortfolioUi, StockDetailScreen, AlertListView – elk een volledig scherm of herbruikbaar UI-onderdeel.
ViewModels	MainViewModel (globale navigatie & loginstate), PortfolioViewModel (portefeuille laden), StockDetailViewModel (metrics & analyse ophalen), SearchViewModel (debounced zoekopdracht).
Modellen	AlertRule, Holding, TransactionRequest, MetricsResponse, MarketAnalysisResponse – weerspiegelen de backend-JSON-contracten.
API	Retrofit + OkHttp. ApiService definieert endpoints. AuthInterceptor vangt 401 af, vernieuwt de token en voert de aanvraag opnieuw uit.
Navigatie	Screen-enum (Market, Portfolio, Profile, Alert) beheerd door MainViewModel. Onderbalk + selectedTicker voor detail-overlay.

Component	Verantwoordelijkheid
Hulpprogramma's	TokenManager (SharedPreferences + biometrische vlag), AlertDataStore (Jetpack DataStore voor alarmregels), BiometricAuthenticator .
Thema	AppTheme (ondersteunt dynamische kleur op Android 12+), aangepaste typografie met Google Fonts ("Azeret Mono", "Italiana"), donkere/lichte schema's.

Backend datastructuur (ERD-beschrijving)

Let op: Holding wordt dynamisch berekend op basis van Transaction-data en bestaat niet als persistente tabel in de database.



AlertRule (Client-side)	StockData (API Proxy)
- id (PK)	- ticker (PK)
- ticker	- metrics (JSON)
- rollingDays	- candles (Array)
- thresholdPercent	- analysis (JSON)
- lastTriggeredAt	- last_fetch
- isEnabled	

- **User** – authenticatie via JWT (access- en refresh-token).
- **Portfolio** – berekend uit de **Transaction**-tabel (som van aankopen minus verkopen). Er is geen aparte "holding"-tabel; holdings worden dynamisch samengesteld.
- **AlertRule** – wordt **client-side** opgeslagen (DataStore) voor offline gebruik en snelheid. Kan later naar de backend worden gesynchroniseerd.
- **StockData** – externe data van financiële API's (koers, ratio's, technische indicatoren). De app vraagt deze aan via een backend-proxy.

Belangrijkste endpoints (gedefinieerd in **ApiService.kt**):

- **POST /api/v1/auth/login** – retourneert {access_token, refresh_token}
- **POST /api/v1/auth/refresh** – accepteert refresh-token, retourneert nieuw access-token
- **POST /api/v1/auth/register** – Nieuwe gebruiker registreren.
- **GET /api/v1/portfolio** – vereist Bearer-token, retourneert holdings + totale waarde
- **POST /api/v1/portfolio/transactions** – voegt een aan- of verkooptransactie toe
- **GET /api/v1/stocks/{ticker}/metrics** – fundamentele data (K/W, winst per aandeel, enz.)
- **GET /api/v1/market/{ticker}/analysis** – technische analyse (RSI, MACD, signalen)
- **GET /api/v1/market/{ticker}/candles/stored** – Historische OHLCV-data voor grafieken
- **GET /api/v1/market/{ticker}/candles/stored** – historische OHLCV-data voor grafieken en alarmberekeningen

Valkuilen – wat je moet weten voordat je bijdraagt

1. **Token refresh interceptor sluit de response** – In `RetrofitClient.AuthInterceptor` roepen we `response.close()` aan **voordat** we de aanvraag opnieuw proberen. Verwijder deze regel niet, anders krijg je `IllegalStateException: cannot make a new request because the previous response is still open`.
2. **AlertWorker gebruikt glijdend venster (geen vaste datums)** – De worker berekent de maximale daling vanaf de **hoogste koers** binnen het opgegeven aantal dagen (`rollingDays`). Dit is bewust gekozen omdat gebruikers een “daling vanaf de piek” verwachten. Pas deze logica niet aan zonder het team te raadplegen.
3. **TokenManager moet worden geïnitieerd** – In `MainActivity.onCreate()` wordt `TokenManager.init(applicationContext)` aangeroepen. Vergeet dit niet in tests of bij refactoring, anders krijg je een `IllegalStateException`.
4. **Biometrische authenticatie vereist FragmentActivity** – `MainActivity` erft over van `FragmentActivity` (niet van `ComponentActivity`). De biometrische prompt heeft de AndroidX-fragmentondersteuning nodig. Behoud deze overerving.
5. **Selectiemodus in portfolio** – Het potloodicoon schakelt `selectionMode` in/uit. Als de selectiemodus uit staat, zijn de “Selecteer alles”-header en de selectievakjes verborgen. Dit is toegevoegd na een gebruikerswens. Voeg je nieuwe acties toe (bijv. geselecteerde aandelen verwijderen), respecteer dan altijd `selectionMode`.
6. **Koop/verkoopdialogen** – Ze gebruiken de **huidige marktprijs** uit de quote als standaardwaarde, maar de gebruiker kan deze overschrijven. De backend verwacht `price_per_share` exact zoals ingevoerd. Er vindt **geen** client-side validatie tegen de echte marktprijs (dit is bewust – gebruikers willen historische transacties tegen een eigen prijs vastleggen).
7. **WorkManager inplannen** – `scheduleStockWorker()` wordt aangeroepen na een succesvolle login (als er een token bestaat). Het plant `AlertWorker` elke 15 minuten. Op Android 13+ wordt toestemming voor notificaties gevraagd. De worker is idempotent (controleert cooldown per regel). Wijzig nooit het interval zonder ook de cooldown-logica aan te passen.
8. **Themakleuren** – Het bestand `ui/theme/Color.kt` is automatisch gegenereerd door de Material Theme Builder. Bewerk het niet handmatig; genereer het thema opnieuw via de [Material Theme Builder](#) en vervang het hele bestand.
9. **AlertRule serialisatie** – `AlertRule` gebruikt `kotlinx.serialization` met een aangepaste `LocalDateSerializer` (hoewel `startDate` en `endDate` niet meer worden gebruikt, blijft de serializer in het bestand staan). Verwijder deze niet zonder migratie van bestaande DataStore-gegevens.



AI-gebruik in dit project

Gebruikte tools & modellen

- **OpenHands** – Volledige backend (FastAPI, databaseschema, endpoints, authenticatie) gegenereerd door OpenHands. De backend draait op een cloudflare tunnel en is **niet** opgenomen in deze repository.
- **DeepSeek** – Gebruikt voor **codereview** van eigen frontend-code en voor het genereren van **boilerplate** (basisstructuren voor composables, ViewModels, repository's).

- **Gemini** – Ingezet voor **codereview** van specifieke complexe stukken (bijv. de **AuthInterceptor**, de **AlertWorker** logica) en voor het aanmaken van **boilerplate** voor dataklassen en utility-functies.

Methodologie & Controle

Het uitgangspunt van dit project is 'human-in-the-loop' ontwikkeling: AI fungeert als een versneller, maar ik behoud de volledige controle over de codebase.

- **Geen AI-gegenereerde frontend:** Hoewel AI ondersteuning bood bij boilerplate, is de volledige frontend-architectuur (UI-code, navigatie, schermen, thema's) handgeschreven. De gegenereerde basis-frontend is niet gebruikt; de UI is vanuit eigen ontwerp opgebouwd.
- **Geen “vibe coding”:** Elke AI-suggestie is kritisch beoordeeld, getest en waar nodig verfijnd. Er is nooit sprake geweest van blind kopiëren; elke regel code is door mij gevalideerd en geïntegreerd binnen de bredere projectcontext.

Concreet AI-gebruik per onderdeel

Onderdeel	AI-bijdrage
Backend (OpenHands)	Volledig gegenereerd – endpoints, databasemodellen, JWT-authenticatie, refresh-token flow. Geen handmatige backend-code geschreven.
RetrofitClient.AuthInterceptor	DeepSeek hielp bij het oplossen van de IllegalStateException door response.close() voor de retry te plaatsen.
AlertWorker logica	Gemini gaf een voorzet voor de “drop from max price” berekening; zelf verder verfijnd.
Boilerplate voor dataklassen	DeepSeek genereerde de eerste versies van AlertRule , TransactionRequest , MetricsResponse (later handmatig aangepast aan de werkelijke API).
AlertDataStore	Gemini leverde het basisskelet voor addRule , updateRule , deleteRule . Zelf de Flow -ondersteuning en serialisatie toegevoegd.
Code review	DeepSeek en Gemini werden gebruikt om eigen code te laten controleren op fouten (bijv. vergeten collectAsState , ontbrekende LaunchedEffect).

Aan de slag voor ontwikkelaars

Vereisten

- Android Studio **Ladybug** (2024.2.1+) of nieuwer
- JDK 17
- Android SDK API 26+ (Android 8.0)

Installatie

1. Kloon de repository

```
git clone https://github.com/sebian-lab/AlphaTracer.git
```

2. Open het project in Android Studio en wacht tot Gradle is gesynchroniseerd.
3. De backend-basis-URL staat hardgecodeerd in `RetrofitClient.BASE_URL` (momenteel een Cloudflare-tunnel). Als je een eigen backend gebruikt, wijzig deze constante.
4. Bouw en run de app op een emulator of fysiek Android-apparaat.

Configuratie

- Geen API-sleutels nodig – de backend proxy verzorgt de externe financiële data.
 - Biometrische authenticatie vereist een apparaat met vingerafdruk/gezicht (of een emulator met virtuele vingerafdruk).
-